Software Requirements Specification

for

Financial Ledger & Digital Wallet API

Version 1.0

Prepared by

Punith M Reddy

Jadav Sai Rugvedh

Laksh Chovatiya

08 November 2026

IIIT Bangalore

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Intended Audience and Reading Suggestions 3
1.4 Product Scope 3
1.5 References 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements 6
3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 6
3.4 Communications Interfaces 7
4. System Features 7
4.1 Wallet Initialization (FR-1) 8
4.2 Idempotent Transfers (FR-2) 8
4.3 Concurrent Balance Mutability (FR-3) 9
4.4 Immutable Audit Trail (FR-4) 10
4.5 Ledger Reconciliation (FR-5) 10
5. Other Nonfunctional Requirements 11
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 12
5.5 Business Rules 12
6. Other Requirements 13
6.1 Data Requirements 13
6.2 Data Retention, Backup and Localization 13
6.3 Skill Requirements 14
6.4 Testing Requirements 14
6.5 Engineering Process Requirements 15
6.6 Traceability Matrix 15
Appendix A: Glossary 16
Appendix B: Analysis Models 17
B.1 Use Case Diagram 17
B.2 Data Model 18
B.3 Transfer Processing Flow 19
B.4 Business Scenarios and Expected Behaviour 19
Appendix C: To Be Determined List 20

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements of the Financial Ledger & Digital Wallet API, version 1.0. It covers the complete backend product: the REST API, the SQLite database, the reconciliation function, the agent skills and the engineering practices defined for the project.

The system exists to remove the "vanishing money" glitch. In poorly designed systems, simultaneous requests overwrite each other's balance updates and a single network retry can charge a user twice. This product treats money as a verifiable record of immutable double-entry ledger entries instead of a number that can be casually overwritten.

1.2 Document Conventions

- Functional requirements are tagged FR-x.y, where x is the project-statement requirement (FR-1 to FR-5) and y is the sequence number inside that feature. Non-functional requirements are tagged NFR-nnn, skill requirements SK-n, testing requirements TQ-n, business rules BR-n, and unresolved items TBD-nn.
- "Shall" marks a mandatory requirement and is a pass/fail acceptance criterion. "Should" marks a recommendation and is not an acceptance criterion. "Must" is not used.
- Priority is stated once per system feature (High, Medium or Low) and is inherited by every requirement of that feature unless a requirement states its own priority.
- Monetary amounts are always integers in minor units (for example paise or cents). Timestamps are ISO 8601 in UTC.
- Terms are used only in the sense defined in Appendix A.

1.3 Intended Audience and Reading Suggestions

This SRS is written for the following readers.

- **Developers:** read Sections 2, 3, 4 and 6, then Appendix B.
- **Testers:** read Sections 4, 5 and 6.4, then Appendix B.4 and the traceability matrix in Section 6.6.
- **Reviewers and course staff:** read Sections 1 and 2 first, then Section 4 and Section 6.6.
- **Security reviewers:** read Sections 3.4, 5.3 and 5.5.
- **Team members maintaining the Second Brain and skills:** read Sections 6.3 and 6.5.

Section 2 gives the overall picture, Section 3 the external interfaces, Section 4 the detailed functional requirements by feature, Section 5 the quality requirements and business rules, and Section 6 the data, skill, testing and process requirements. The appendices hold the glossary, analysis models and the list of open items.

1.4 Product Scope

The product is a backend for a digital wallet. It lets clients create users and single-currency wallets, fund wallets, transfer money between wallets safely under retries and concurrency, reverse transfers, and verify the whole ledger through an administrative reconciliation endpoint. The product's objectives are:

- every balance is explained by double-entry ledger entries;
- a retried transfer never charges an account twice;
- concurrent requests never corrupt a balance;
- ledger history can never be rewritten, only corrected by explicit reversals; and

- correctness is demonstrated by automated tests and agent skills.

Out of scope for this release: cross-currency transfers and exchange rates, interest and fees, real payment gateways or bank integrations, real money or real personal financial data, customer-facing web or mobile applications, per-user authentication, and distributed databases or lock servers.

1.5 References

- Project Statement: Financial Ledger & Digital Wallet API (Project 2), course material.
- The AI-Assisted Software Development Life Cycle (SDLC) and Workflow document, course material.
- Software Requirements Specification template (IEEE format), supplied by the course instructor.
- SQLite documentation: transactions, locking and WAL mode, https://www.sqlite.org/docs.html.
- RFC 9110, HTTP Semantics; ISO 4217, Currency codes; ISO 8601, Date and time format.
- Agent Skills format specification, https://agentskills.io.
- OpenAPI Specification 3.x, https://spec.openapis.org.

2. Overall Description

2.1 Product Perspective

The product is a new, self-contained backend service with no predecessor. It is one deployable service that owns one SQLite database file. It does not depend on any external system at run time. Clients (Postman, Bruno, cURL or a future application) and administrators reach it only through its HTTP interface. Figure 1 shows the major components.

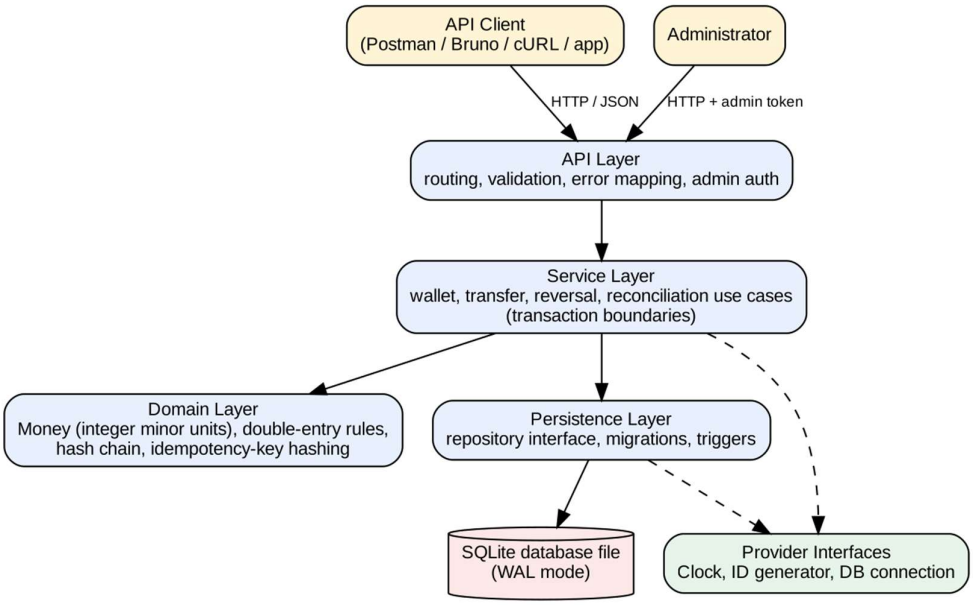


Figure 1: Major components and external interfaces

All external dependencies (clock, identifier generation and database connection) sit behind provider interfaces so that tests can substitute fakes and the database could later be replaced (SE-6).

2.2 Product Functions

- Create users and per-currency wallets (FR-1).
- Fund a wallet through a balanced deposit from a system funding account (FR-1).
- Transfer funds between two wallets with an idempotency key (FR-2).
- Process simultaneous requests on one wallet without balance corruption (FR-3).
- Record every movement as immutable, hash-chained double-entry ledger entries and correct mistakes by reversal (FR-4).
- Reconcile the ledger against wallet balances through an administrative endpoint (FR-5).
- Provide agent skills: simulate-transfer, local-check, brain-ingest, brain-refine and brain-query (Section 6.3).

Figure 2 in Appendix B.1 shows the grouping of these functions by actor.

2.3 User Classes and Characteristics

User class	Characteristics	Functions used	Importance
API Client	An application or tester acting for wallet owners. May retry any request at any time. Technical.	Create user and wallet, deposit, transfer, reverse, read balance and ledger	Primary
Administrator	Privileged operator holding the admin credential. Cannot edit or delete ledger entries.	Run reconciliation	Primary
Developer / Team Member	Builds, tests and runs the project locally and in CI; uses agent skills and the Second Brain.	local-check, simulate-transfer, brain skills, test suite	Primary
AI Coding Assistant	Generates changes under the AI-assisted development policy; every change is human-reviewed.	Source, skills and brain changes via pull requests	Secondary
Evaluator	Course staff reviewing documents and the final demonstration.	Read documents, watch demo	Secondary

2.4 Operating Environment

- Development on Linux, macOS or Windows; CI on Linux runners (GitHub Actions).
- SQLite 3.35 or later, as a single file on disk for development and in memory or a fresh temporary file for tests.
- HTTP/1.1 clients such as Postman, Bruno or cURL.
- A single service process per database file. No network dependency other than the HTTP interface.

2.5 Design and Implementation Constraints

- **C-1:** SQLite shall be the database, with WAL mode (PRAGMA journal_mode=WAL) and foreign keys enabled. No distributed lock server is used.
- **C-2:** All monetary values shall be integers in minor units. Floating-point types shall not hold money.
- **C-3:** The interface shall be REST over JSON and shall be documented in OpenAPI.
- **C-4:** External dependencies shall be isolated behind provider interfaces (SE-6).

- **C-5:** Every change, including code, skills and brain notes, shall go through a pull request with at least one human review.
- **C-6:** Tests and CI shall use an isolated database, never a shared or production file.
- **C-7:** The implementation language and framework shall be chosen and recorded in an ADR (TBD-01).

2.6 User Documentation

- README with install, test and run steps.
- Developer API documentation as an OpenAPI file, kept in step with the implementation.
- User guide with cURL examples for each of FR-1 to FR-5.
- AGENTS.md at the repository root.
- Test documentation: test plan, results per stage, skill evaluation results and defect log.
- Second Brain index (brain/index.md) and run log (brain/log.md).

2.7 Assumptions and Dependencies

- Each deployment runs one service process against one database file; SQLite serialises writers.
- Server-side timestamps are authoritative; client timestamps are never trusted.
- Clients reuse the same Idempotency-Key when retrying one logical operation and use a new key for each new operation.
- Every transfer is between two wallets of the same currency.
- Each supported currency has one system funding account created at startup.
- The admin credential is supplied through configuration and never stored in the repository.
- GitHub and GitHub Actions remain available to the team.

3. External Interface Requirements

3.1 User Interfaces

The product has no graphical user interface. The only interface is the REST API, exercised through Postman, Bruno or cURL. Error responses shall follow one JSON envelope containing a machine-readable code, a human-readable message and a request id. A demonstration client or UI is optional and is not an acceptance criterion.

3.2 Hardware Interfaces

The product has no hardware interfaces beyond local disk storage for the SQLite database file and its WAL files. The data directory shall be writable by the service process.

3.3 Software Interfaces

Component	Version	Purpose and data exchanged
SQLite	3.35 or later	Persistence of users, wallets, ledger and idempotency records; transactions and triggers
GitHub / GitHub Actions	Current	Issues, pull requests and CI (build, lint, tests, secret scanning, SKILL.md and brain validation)
Obsidian vault (brain/)	Current	Second Brain notes in PARA layout, linked with wikilinks
Agent runtime	Any compliant	Executes SKILL.md skills against the API and repository

Component	Version	Purpose and data exchanged
API testing tools	Postman, Bruno, cURL	Manual and scripted API verification

3.4 Communications Interfaces

Clients communicate over HTTP/1.1 using JSON in UTF-8 under the base path /v1. Any deployment beyond local use shall sit behind TLS. Ledger content is not confidential from the service operator, so end-to-end encryption is not required. No synchronisation mechanism other than idempotency keys is needed.

Method and path	Purpose	Idempotency-Key	Auth
GET /health	Liveness check, read-only	No	None
POST /v1/users	Create user	No	None (open)
POST /v1/wallets	Create wallet for a user and currency	No	None (open)
GET /v1/wallets/{id}	Wallet and current balance	No	None (open)
GET /v1/wallets/{id}/entries	Ledger entries of a wallet	No	None (open)
POST /v1/wallets/{id}/deposits	Fund wallet from the system funding account	Required	None (open)
POST /v1/transfers	Transfer between two wallets	Required	None (open)
GET /v1/transfers/{id}	Transfer details with its entries	No	None (open)
POST /v1/transactions/{id}/reversal	Reverse a transfer or deposit	Required	None (open)
GET /v1/admin/reconciliation	Reconciliation report	No	Admin

In this release client endpoints are open: there is no per-user authentication (see TBD-07). Only /v1/admin/* requires the admin credential; /health needs none.

Status codes: 200 and 201 success; 400 malformed input; 401 missing or invalid credential (403 is reserved for scoped access, TBD-07); 404 unknown resource; 409 conflict; 422 semantically invalid request (including insufficient funds and reuse of an idempotency key with a different payload); 503 database busy. Request headers: Idempotency-Key, X-Request-ID and the admin credential. Response headers: Idempotent-Replayed, X-Request-ID and Retry-After.

4. System Features

The functional requirements are organised by system feature. The five features correspond one-to-one with FR-1 to FR-5 of the project statement.

4.1 Wallet Initialization (FR-1)

4.1.1 Description and Priority

Create users and distinct per-currency wallets, and give wallets funds only through balanced ledger entries, so that every balance is backed by double-entry records. **Priority: High.** Ratings (1 low to 9 high): benefit 9, penalty 9, cost 5, risk 5.

4.1.2 Stimulus/Response Sequences

- Stimulus: the client posts a user. Response: 201 with the new user id; 409 if the email address is already in use; 400 on invalid input.
- Stimulus: the client posts a wallet (user id, currency). Response: 201 with the wallet at balance 0; 409 if the user already has a wallet in that currency; 400 or 422 on invalid input.
- Stimulus: the client posts a deposit with an Idempotency-Key. Response: 201 with the new balance after one balanced transaction; invalid input changes nothing.

4.1.3 Functional Requirements

FR-1.1: The system shall create a user with a system-generated UUID and a unique email address; a request whose email address is already in use shall return 409 and create nothing.

FR-1.2: The system shall allow at most one wallet per user and ISO 4217 currency code; a second request for the same pair shall return 409.

FR-1.3: A new wallet shall have a balance of 0 minor units.

FR-1.4: The API shall not offer any operation that sets or overwrites a balance directly.

FR-1.5: A deposit shall record two ledger entries in one transaction: a debit on the system funding account of the currency and a credit on the destination wallet, for the same amount.

FR-1.6: Every ledger transaction shall have total debits equal to total credits; a transaction that violates this shall be rejected and rolled back.

FR-1.7: Identifiers shall be UUIDs; a malformed identifier shall return 400.

FR-1.8: Amounts shall be positive integers in minor units; zero, negative, fractional or non-numeric amounts shall return 400 and store nothing.

FR-1.9: A currency code outside the supported list (INR, USD and EUR in this release, TBD-02) shall return 422 and store nothing.

4.2 Idempotent Transfers (FR-2)

4.2.1 Description and Priority

Move funds between two wallets so that a retried network request never charges an account twice. **Priority: High.** Ratings (1 low to 9 high): benefit 9, penalty 9, cost 5, risk 5.

4.2.2 Stimulus/Response Sequences

- Stimulus: new Idempotency-Key and valid body. Response: the transfer executes and 201 is returned; the response is stored against the key.
- Stimulus: repeated key with an identical body. Response: the original status and body are returned with Idempotent-Replayed: true; nothing new is written.
- Stimulus: repeated key with a different body. Response: 422 and no change.
- Stimulus: source balance lower than the amount. Response: 422 INSUFFICIENT_FUNDS and no change.

4.2.3 Functional Requirements

FR-2.1: POST /v1/transfers, deposits and reversals shall require an Idempotency-Key header of 1 to 64 characters from [A-Za-z0-9_-]; a missing or invalid key shall return 400.

FR-2.2: The transfer body shall contain exactly source_wallet_id, destination_wallet_id, amount_minor and currency; unknown or missing fields shall return 400.

FR-2.3: The system shall compute a SHA-256 hash of the HTTP method, the request path and the canonical form of the request body (keys sorted, no whitespace) and store it with the key, so that one key cannot be replayed against a different operation or resource.

FR-2.4: A repeated key with the same hash shall return the original status code and body without writing new ledger entries and shall add the header Idempotent-Replayed: true.

FR-2.5: A repeated key with a different hash shall return 422 and write nothing.

FR-2.6: The idempotency record and the ledger entries of a transfer shall be written in the same database transaction, so a crash cannot leave one without the other.

FR-2.7: Idempotency keys shall be retained for at least 24 hours (TBD-03).

FR-2.8: Two concurrent requests with the same key shall result in exactly one execution; the request that obtains the write lock second shall return the stored result of the first (with Idempotent-Replayed: true), or 503 with Retry-After if the lock times out (FR-3.4).

FR-2.9: Source and destination wallets shall differ and shall both use the transfer currency; otherwise the system shall return 422. An unknown wallet shall return 404.

FR-2.10: A transfer exceeding the source balance shall return 422 with code INSUFFICIENT_FUNDS and write nothing.

FR-2.11: A successful transfer shall write one debit entry on the source wallet and one credit entry on the destination wallet in a single transaction.

FR-2.12: A failure after the transaction starts shall leave wallets, ledger and idempotency table unchanged, and a retry with the same key shall be able to succeed.

FR-2.13: Only a successfully committed result (2xx) shall be stored against an idempotency key. A request rejected with a 4xx or 5xx error shall leave no idempotency record, so the client may retry with the same key once the cause is corrected.

4.3 Concurrent Balance Mutability (FR-3)

4.3.1 Description and Priority

Handle multiple simultaneous incoming and outgoing transfers on the same wallet without data races or balance corruption. **Priority: High.** Ratings (1 low to 9 high): benefit 9, penalty 9, cost 5, risk 5.

4.3.2 Stimulus/Response Sequences

- Stimulus: several requests touch one wallet at the same moment. Response: the database serialises them; each completes fully or is rejected cleanly.
- Stimulus: two requests each try to spend most of the same balance. Response: exactly one succeeds and the other returns 422 INSUFFICIENT_FUNDS.
- Stimulus: the write lock cannot be obtained in time. Response: 503 with Retry-After and no change.

4.3.3 Functional Requirements

FR-3.1: Every state-changing request shall execute as one ACID transaction started with BEGIN IMMEDIATE.

FR-3.2: The balance check and the balance update shall occur inside the same transaction, never as separate read and write steps.

FR-3.3: The service shall use WAL mode and a busy timeout (default 5000 ms, TBD-04).

FR-3.4: If the lock is not acquired within the timeout the service shall return 503 with a Retry-After header and write nothing.

FR-3.5: Given a wallet with balance B and N concurrent debits, the total successfully debited shall not exceed B and the final balance shall equal B minus the sum of successful debits.

FR-3.6: A user wallet balance shall never become negative.

FR-3.7: Transfers in opposite directions between the same two wallets shall not deadlock.

FR-3.8: Each request shall use its own database connection; connections shall not be shared between threads.

4.4 Immutable Audit Trail (FR-4)

4.4.1 Description and Priority

Keep ledger records append-only. Corrections and cancellations are recorded as explicit reversal transactions, and tampering is detectable. **Priority: High.** Ratings (1 low to 9 high): benefit 9, penalty 9, cost 5, risk 5.

4.4.2 Stimulus/Response Sequences

- Stimulus: any successful money movement. Response: new ledger rows are inserted; none is modified.
- Stimulus: the client requests a reversal with an Idempotency-Key. Response: 201 with a new transaction that mirrors the original; the original is unchanged.
- Stimulus: a second reversal of the same transfer. Response: 409.
- Stimulus: a direct SQL UPDATE or DELETE on a ledger row. Response: the database aborts the statement.

4.4.3 Functional Requirements

FR-4.1: Ledger entries and ledger transactions shall not be updatable or deletable; database triggers shall abort any UPDATE or DELETE on them.

FR-4.2: The API shall expose no operation that edits or removes ledger records.

FR-4.3: Each ledger entry shall record sequence number, transaction id, wallet id, direction (DEBIT or CREDIT), amount_minor, currency and a server-generated UTC timestamp.

FR-4.4: Each ledger entry shall store prev_hash and entry_hash, where entry_hash is the SHA-256 of prev_hash concatenated with the fields of the entry, forming a tamper-evident chain.

FR-4.5: POST /v1/transactions/{id}/reversal shall create a new transaction with swapped debit and credit legs and a reference to the original transaction. {id} is the id of a transfer or deposit transaction, as returned in the response that created it.

FR-4.6: A transfer or deposit shall be reversible at most once and only in full; a second reversal, or a reversal of a reversal, shall return 409.

FR-4.7: A reversal that would make the balance of the wallet being debited negative (the transfer destination, or the funded wallet in the case of a deposit) shall return 422 with code INSUFFICIENT_FUNDS.

FR-4.8: GET /v1/wallets/{id}/entries shall return the entries of the wallet in ascending sequence order using cursor-based pagination of at most 100 entries per page (TBD-08).

4.5 Ledger Reconciliation (FR-5)

4.5.1 Description and Priority

Provide an administrative endpoint that scans ledger entries and verifies that credits equal debits and that wallet balances match the calculated ledger sums. **Priority: High.** Ratings (1 low to 9 high): benefit 9, penalty 9, cost 5, risk 5.

4.5.2 Stimulus/Response Sequences

- Stimulus: administrator calls GET /v1/admin/reconciliation with the admin credential. Response: 200 with a PASS or FAIL report.
- Stimulus: call without valid credentials. Response: 401 and nothing is run.

4.5.3 Functional Requirements

FR-5.1: The endpoint shall require the admin credential and shall return 401 otherwise.

FR-5.2: The check shall verify that, for every transaction and for the whole ledger, the sum of debits equals the sum of credits.

FR-5.3: The check shall verify that each wallet's stored balance equals the sum of its credits minus the sum of its debits.

FR-5.4: The check shall verify the hash chain and report the first sequence number at which it breaks.

FR-5.5: The response shall contain the overall status (PASS or FAIL), counts of transactions, entries and wallets checked, and a list of discrepancies giving wallet id, transaction id, expected value and actual value (empty on PASS).

FR-5.6: The check shall be read-only and shall never edit or delete ledger data.

FR-5.7: The check shall run in a single read transaction so that concurrent transfers do not cause false alarms.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Targets apply to a local run on a typical developer laptop with SQLite in WAL mode. They exist so that correctness mechanisms (locking, hashing, triggers) are not tuned away.

NFR-001: 95% of transfer requests shall complete within 200 ms at 50 concurrent clients.

NFR-002: Reconciliation of 100,000 ledger entries shall complete within 10 seconds.

NFR-003: A load test of 100 concurrent transfers on one wallet shall finish with zero integrity violations, shown by a PASS reconciliation afterwards.

5.2 Safety Requirements

Harm in this product means loss, creation or duplication of money.

NFR-004: Money shall be created or destroyed only through balanced transactions involving the system funding account.

NFR-005: No retry, crash or concurrent request shall cause a double charge or a lost update.

NFR-006: Arithmetic that would overflow a 64-bit signed integer shall be rejected before execution; the maximum single amount shall be 10^12 minor units (TBD-05).

NFR-007: After a process crash or power loss, the database shall reopen with all committed transactions present and no partially applied transaction.

5.3 Security Requirements

NFR-008: All API parameters (path, query, header and body) shall be validated for type, range, length and format before use.

NFR-009: Negative, zero and non-integer amounts shall be rejected so that no caller can inflate a balance.

NFR-010: All SQL shall use parameterised statements.

NFR-011: Administrative endpoints shall be protected by an admin credential read from configuration or environment variables; it shall never be committed to the repository (TBD-06).

NFR-012: Error responses shall not expose stack traces, SQL or file paths.

NFR-013: Logs shall not contain the admin credential or full request bodies, and test data shall contain no real personal data.

NFR-014: CI shall run secret scanning on every pull request, and no secrets, tokens or credentials shall be pasted into any AI tool.

Per-user authentication and scoped access to wallets are out of scope for this release and shall be specified before any non-local deployment (TBD-07).

5.4 Software Quality Attributes

Attribute	Requirement	Verified by
Correctness	NFR-015: Business rules BR-1 to BR-5 shall hold after every operation.	TQ-1, TQ-3, TQ-4
Reliability	NFR-016: With synchronous mode set to FULL (or NORMAL with WAL, justified in an ADR), the recovery point is the last committed transaction. NFR-017: The service shall restart and accept requests within 5 minutes of a failure.	TQ-2
Recoverability	NFR-018: Backups shall use SQLite's online backup mechanism, and a restore shall be followed by a reconciliation run returning PASS.	Test plan
Testability	NFR-019: Tests shall use an isolated in-memory or fresh temporary database, and time and ID providers shall be fakeable.	TQ-5
Maintainability	NFR-020: Code shall follow the layers in 2.1, pass the linter in CI and reach at least 80% statement coverage in the domain and service layers. NFR-021: Every significant decision shall be an ADR citing a brain note.	CI, review
Usability (API)	NFR-022: All endpoints shall share one naming and error-envelope convention. NFR-023: An OpenAPI document shall describe every endpoint, header, schema and error code and shall match the implementation.	CI check
Portability	NFR-024: A new member shall be able to install, test and start the server on Linux, macOS or Windows from the README in under 15 minutes.	local-check
Scalability	NFR-025: The product is scoped to one node. The repository interface shall allow a later change of database engine without changing domain code.	Design review
Observability	NFR-026: Each request shall carry a request id in logs and error responses.	Review

Relative preference: correctness and integrity are valued above performance, and performance above convenience.

5.5 Business Rules

These ledger invariants are operating principles of the product. Design and tests shall preserve all of them.

BR-1: Every ledger transaction's entries balance: total debits equal total credits within one currency.

BR-2: A wallet's balance equals the sum of its credits minus the sum of its debits.

BR-3: The sum of all wallet balances in a currency, system funding account included, is always zero.

BR-4: Ledger entries are append-only: a committed entry never changes and is never deleted.

BR-5: A user wallet's balance is never negative. The system funding account is the only wallet allowed to go negative.

BR-6: A user has at most one wallet per currency, and a transfer is always within one currency.

BR-7: Only an administrator may run reconciliation, and no role, administrators included, may edit or delete ledger records.

BR-8: When several checks fail, the API reports the first in this order: authentication, header format, body validation, idempotency (replay or conflict), wallet existence, currency match, available balance. This is the order shown in Figure 4.

6. Other Requirements

6.1 Data Requirements

Entity	Minimum fields	Key constraints
users	id, email, name, created_at	id primary key; email unique
wallets	id, user_id, currency, balance_minor, is_system, created_at	Unique (user_id, currency); foreign key to users; balance never below 0 unless is_system
ledger_transactions	id, type (deposit, transfer, reversal), amount_minor, currency, reverses_transaction_id, created_at	amount_minor positive; reverses_transaction_id unique so a transfer is reversed at most once
ledger_entries	seq, transaction_id, wallet_id, direction, amount_minor, currency, prev_hash, entry_hash, created_at	Foreign keys; amount_minor positive; direction check; triggers block UPDATE and DELETE
idempotency_keys	key, request_hash, response_status, response_body, transaction_id, created_at	key primary key (enforced by database, not only by application code)

The full schema and ER diagram are finalised in Stage 2 (Appendix B.2 shows the current model). A cached wallet balance is kept in step with the ledger inside the same transaction; reconciliation checks it.

6.2 Data Retention, Backup and Localization

- Ledger transactions and entries shall be retained for the lifetime of the database and never deleted.
- Idempotency keys shall be retained for at least 24 hours; expired keys may be purged and the purge shall never touch ledger tables.
- Test databases shall be created fresh and discarded after each CI run.
- The API exchanges only minor units and ISO 4217 codes; display formatting is a client concern. Timestamps are ISO 8601 UTC.
- Legal: the product is an educational prototype and shall hold no real money or real personal financial data.

6.3 Skill Requirements

Each skill is a folder with a SKILL.md in the agentskills.io format. Per SB-3 every skill cites at least one distilled brain note, and per TQ-7 every skill is evaluated before each stage submission.

ID	Skill	Requirement
SK-1	simulate-transfer	Run a complete product workflow and verify ledger consistency: (1) create two test wallets, (2) record initial balances, (3) execute a transfer with an idempotency key, (4) fire two concurrent transfer requests to test race prevention, (5) call the reconciliation endpoint and summarise final balances. It shall use only test data and an isolated database, report every check, never hide a failed step, size the two concurrent requests so only one can succeed, and show expected balances, actual balances and reconciliation status.
SK-2	local-check	Confirm the project works on a team member's machine: (1) build, (2) run the automated test suite, (3) start the local server, (4) invoke one read-only endpoint (GET /health), (5) report pass or fail for each step. It shall stop at the first failure and exit non-zero on failure.
SK-3	brain-ingest	Add new material or summaries to brain/inbox/ with source links and frontmatter, link them to requirement IDs and log the run in brain/log.md. Used to produce at least 10 research notes in Stage 1.
SK-4	brain-refine	Move inbox notes into the PARA folders, merge duplicates, repair broken wikilinks, update brain/index.md and log the run.
SK-5	brain-query	Answer questions from the vault with citations to notes and draft documents such as the SRS and ADRs.
SK-6	All skills	Every skill shall have a SKILL.md whose frontmatter and name pass CI validation, shall cite a brain note, and shall be evaluated before each stage submission. Each brain skill shall write a line to brain/log.md and deliver its changes as a pull request (SB-5).

6.4 Testing Requirements

ID	Level	Requirement
TQ-1	Unit	Accounting calculations, ledger balance assertions (BR-1 to BR-5), idempotency-key hashing and amount validation.
TQ-2	Integration	Database transactions commit as one unit; rollback on simulated transfer failures leaves wallets, ledger and idempotency tables unchanged; constraints and triggers fire.
TQ-3	System (end to end)	At least one test for each of FR-1 to FR-5 through the HTTP API, including the full transfer, reversal and reconciliation flow.
TQ-4	Negative and concurrency	Negative and zero amounts, insufficient funds, invalid idempotency payloads, malformed IDs, missing wallets, currency mismatch, double reversal, and simultaneous concurrent transfers proving race safety.
TQ-5	CI	All tests run against an isolated in-memory or fresh database on every push and pull request.
TQ-6	Precision	Amounts are handled strictly as integer minor units or exact decimals; floats and fractional JSON numbers are rejected; the sum of 1,000 transfers of 1 minor unit is exact.
TQ-7	Skills	Skill evaluations run before each stage submission and results are recorded in the test documentation.
TQ-8	Maintenance	Tests are updated as the system changes; every significant defect gets a regression test.

ID	Level	Requirement
Security	Security	Input validation on all parameters, negative-amount prevention, admin endpoint rejection of missing or wrong credentials, no internal details in errors, no secrets in the repository.

6.5 Engineering Process Requirements

- **Second Brain (SB-1 to SB-6):** an Obsidian vault in brain/ using PARA (inbox, projects, areas, resources, archive) with index.md and log.md; notes linked with [[wikilinks]] and to requirement IDs, issues and PRs; a short review note at the end of each stage; AGENTS.md at the repository root; all brain changes through pull requests; notes authored by every member.
- **Software engineering (SE-1 to SE-7):** this SRS, design documents with UML and ER diagrams, ADRs in docs/adr/ citing brain notes, a development plan and task breakdown, a traceability matrix, provider interfaces for external dependencies, and user and API documentation.
- **GitHub workflow:** a documented branching strategy (trunk with short-lived feature branches, TBD-09), meaningful incremental commits from every member, GitHub Issues for all work and defects, at least one reviewer on every pull request, release tags and a changelog.
- **CI on every pull request:** build, lint, automated tests, secret scanning, validation of every SKILL.md and validation of brain notes (schema and broken links).
- **AI-assisted development policy:** a human reviews every AI-generated change; every member can explain any merged artifact; significant AI sessions are recorded as ai-session notes; no secrets are pasted into AI tools.
- **Stage deliverables:** Stage 1 Requirements (this SRS, brain vault with at least 10 research notes, AGENTS.md, repository and CI skeleton); Stage 2 Design; Stage 3 Testing and CI; Stage 4 Review with a live demonstration of FR-1 to FR-5, simulate-transfer, local-check and brain-query, and a final release tag.

6.6 Traceability Matrix

Issue and pull request numbers are added as GitHub issues are created, completing the chain required by SE-5. Brain note names shown are planned.

Requirement	Issue / PR	Tests	Skill	Brain note (planned)
FR-1.x	#TBD	TQ-1 to TQ-4	SK-1	[[double-entry-bookkeeping]]
FR-2.x	#TBD	TQ-1 to TQ-4	SK-1	[[idempotency-keys]]
FR-3.x	#TBD	TQ-2, TQ-4	SK-1	[[sqlite-wal-and-locking]]
FR-4.x	#TBD	TQ-2 to TQ-4	SK-1	[[immutable-ledger-and-reversals]]
FR-5.x	#TBD	TQ-3	SK-1	[[ledger-reconciliation]]
NFR-006, NFR-008, NFR-009	#TBD	TQ-4, TQ-6	SK-2	[[integer-money]], [[api-input-validation]]
NFR-020, NFR-023	#TBD	CI lint, OpenAPI check	SK-2	[[project-structure]]
SK-1 to SK-6	#TBD	TQ-7, CI validation	SK-1 to SK-5	[[para-second-brain]]

Appendix A: Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability: the transaction guarantees of the database.
Administrator	A privileged operator holding the admin credential.
API	Application Programming Interface; here, the REST interface of the product.
ADR	Architecture Decision Record, stored in docs/adr/.
Double-entry	The rule that every transaction posts debits and credits that sum to zero in one currency.
Hash chain	Ledger entries linked by hashes of their predecessors, so any later change is detectable.
Idempotency key	A client-chosen header value that makes retries of one operation safe.
Ledger entry	One immutable debit or credit line against one wallet.
Ledger transaction	A group of ledger entries created together that must balance.
Minor unit	The smallest unit of a currency (paisa, cent); amounts are integers in minor units.
PARA	Projects, Areas, Resources, Archive: the note organisation used by the Second Brain.
Reconciliation	Recomputing balances from the ledger and comparing them with stored values and checking that credits equal debits.
Reversal	A new transaction that offsets an earlier one without altering it.
Second Brain	The Obsidian knowledge base in brain/ that records the team's distilled knowledge.
SRS	Software Requirements Specification.
System funding account	An internal per-currency account that is the counterparty of deposits and the only wallet allowed to go negative.
User	A person or application identity that owns wallets.
Wallet	A single-currency account owned by a user whose balance is derived from ledger entries.
WAL	Write-ahead logging, a SQLite journal mode that allows readers during a write.

Appendix B: Analysis Models

B.1 Use Case Diagram

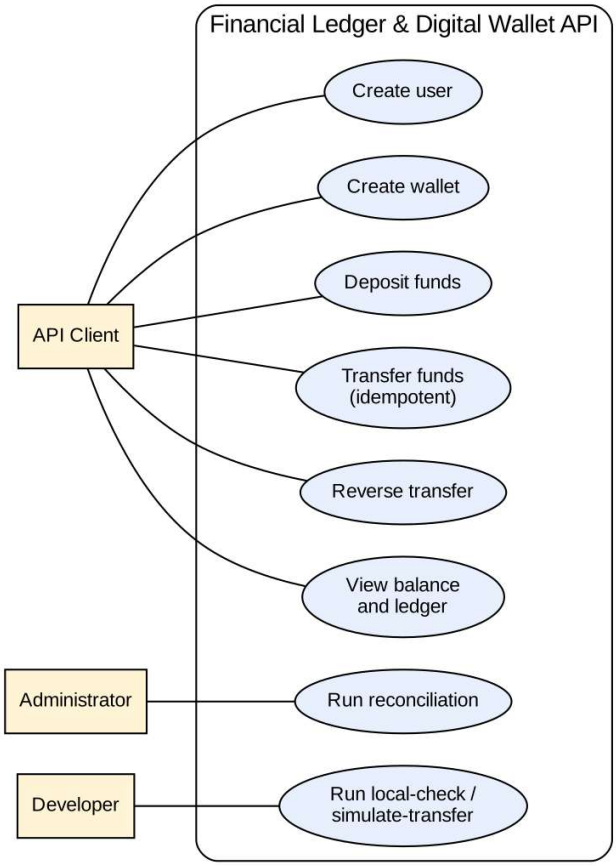


Figure 2: Use cases by actor

B.2 Data Model

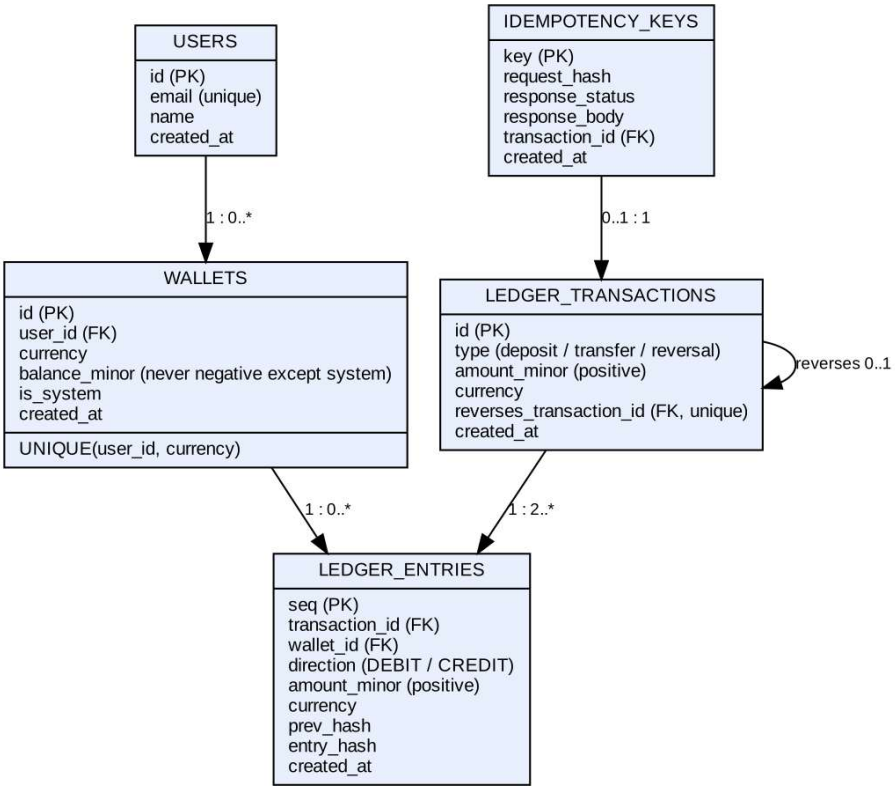


Figure 3: Entity-relationship model

B.3 Transfer Processing Flow

The sequence below applies to every state-changing request that carries an Idempotency-Key.

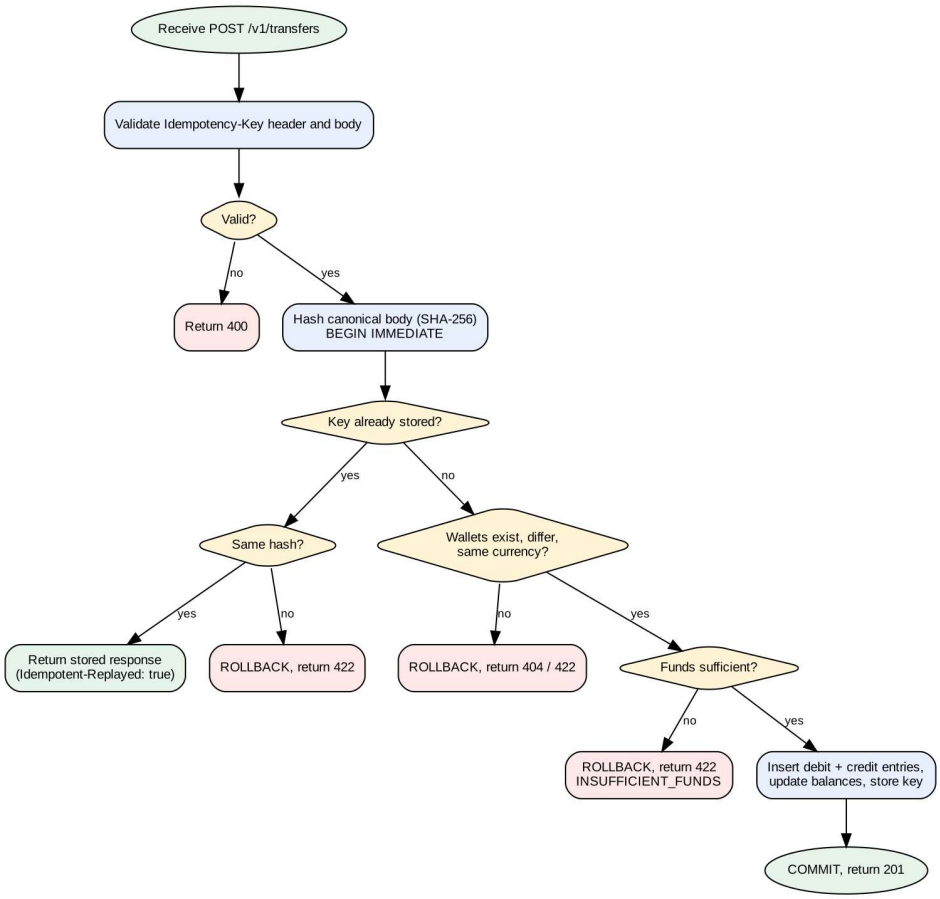


Figure 4: Idempotent transfer processing flow

On any error before commit the transaction rolls back, including the idempotency record, so the client can retry with the same key.

B.4 Business Scenarios and Expected Behaviour

ID	Scenario	Expected behaviour
E.01	Transfer request without Idempotency-Key	400, no writes
E.02	Same key and same body sent twice	Same response, one set of entries
E.03	Same key with a changed amount	422, no writes
E.04	Two simultaneous requests with the same key	One execution; the other replays the stored result
E.05	Amount is -500	400, no writes
E.06	Amount is 10.50 or a string	400, no writes
E.07	Source balance below the amount	422 INSUFFICIENT_FUNDS
E.08	Source equals destination, or currencies differ	422
E.09	Wallet id is not a UUID / wallet does not exist	400 / 404
E.10	Failure injected after the debit insert	Full rollback, balances unchanged
E.11	100 parallel debits on one wallet	Balance never negative; reconciliation PASS
E.12	Direct UPDATE or DELETE on a ledger row	Rejected by trigger

ID	Scenario	Expected behaviour
E.13	Reverse the same transfer twice	Second call returns 409
E.14	Reverse a transfer or deposit when the wallet to be debited lacks funds	422 INSUFFICIENT_FUNDS
E.15	Ledger row altered directly in the database file	Reconciliation FAIL with the broken sequence number
E.16	Stored wallet balance altered directly	Reconciliation FAIL with wallet id and difference
E.17	Reconciliation without admin credential	401, nothing run
E.18	Database locked beyond the busy timeout	503 with Retry-After, no writes

Appendix C: To Be Determined List

ID	Item	Proposed default
TBD-01	Implementation language and framework	Decided by ADR in Stage 2
TBD-02	Supported currencies	INR, USD, EUR
TBD-03	Idempotency key retention period	24 hours
TBD-04	SQLite busy timeout and retry count	5000 ms, bounded retries
TBD-05	Maximum single amount	10^12 minor units
TBD-06	Admin authentication scheme	Static token from an environment variable
TBD-07	Client authentication and wallet scoping	Out of scope in this release; required before non-local deployment
TBD-08	Pagination style for ledger entries	Cursor-based, 100 per page
TBD-09	Branching strategy	Trunk with short-lived feature branches